

REPORT TECNICO DI CYBERSECURITY

Analisi Sperimentale e Mitigazione di un Attacco DoS Volumetrico (UDP Flood)

1. Executive Summary

Nell'ambito dell'attività di laboratorio dedicata allo studio degli attacchi di tipo **Denial of Service (DoS)**, è stata eseguita una simulazione controllata di **UDP Flood** (Layer 4 del modello OSI).

L'obiettivo dell'esercitazione è analizzare il comportamento di un sistema target (*Metasploitable 2*) quando sottoposto a un traffico ad alta frequenza di datagrammi UDP diretti a una porta non attiva, valutando l'impatto prestazionale sul sistema e documentando le relative contromisure difensive (Blue Teaming) tramite regole di firewalling e firme di rilevamento IDS.

L'attività si è svolta in un ambiente virtuale isolato, garantendo la totale sicurezza dei sistemi esterni. L'analisi dei risultati dimostra che un flusso di soli **34,12 Mbps** è in grado di saturare le risorse di calcolo della CPU del target a causa dell'overhead generato dalla gestione delle risposte ICMP di errore.

2. Architettura del Laboratorio e Topologia di Rete

Per garantire l'isolamento della simulazione, le macchine virtuali sono state collegate a una rete privata isolata (*Host-Only*), definita sulla subnet 192.168.50.0/24.

- **Host Attaccante (Kali Linux):** 192.168.50.150
- **Host Target (Metasploitable 2):** 192.168.50.101
- **Porta Target:** 5005 (Protocollo UDP - Stato: Chiuso)
- **Strumenti di telemetria:**
 - **Wireshark v4.x** (in esecuzione su Kali Linux) per la cattura e l'analisi dei pacchetti.
 - **Utility top** (in esecuzione su Metasploitable) per il monitoraggio della CPU in tempo reale.

3. Implementazione del Software di Simulazione

In conformità con i requisiti didattici tracciati, è stato sviluppato uno script in linguaggio Python 3 che adotta un approccio interattivo e concorrente (multi-threaded).

Risoluzione dei requisiti di progetto:

1. **Input dell'IP Target:** Gestito dinamicamente all'avvio del programma tramite prompt interattivo.
2. **Input della Porta Target:** Acquisito dall'utente e validato nel range di porte valido (0–65535).
3. **Costruzione del Pacchetto (1 KB):** Implementata mediante la funzione crittografica `random._urandom(1024)`. La generazione di byte ad alta entropia garantisce che il payload non presenti pattern ripetitivi, simulando traffico reale non comprimibile dagli apparati di rete.
4. **Numero di Pacchetti:** Richiesto in input all'utente. Il carico complessivo viene distribuito equamente su 4 thread paralleli tramite la libreria `threading`. L'uso del multi-threading ottimizza le chiamate di sistema (`syscall`) sul socket, consentendo di superare i limiti prestazionali di un singolo thread concorrente.

4. Analisi dei Risultati e Metriche Sperimentali

Il test è stato avviato inserendo come input l'IP del target (192.168.50.101), la porta destinazione (5005) e un volume di **100.000 pacchetti complessivi**. Lo script ha completato la trasmissione generando le seguenti metriche:

- **Pacchetti trasmessi:** 100.000 su 100.000 (100% di successo)
- **Tempo totale di esecuzione:** 22,8975 secondi
- **Volume dati complessivo:** 97,66 MB
- **Velocità media di trasmissione:** 4.367,29 pacchetti al secondo (PPS)
- **Larghezza di banda media:** 34,12 Mbps

Plaintext

=====

RISULTATI E METRICHE DI CARICO

=====

[*] Pacchetti inviati con successo: 100,000 su 100,000
[*] Tempo impiegato: 22.8975 secondi
[*] Volume totale dei dati inviati: 97.66 MB
[*] Velocità media di invio: 4367.29 pacchetti/secondo
[*] Larghezza di banda teorica: 34.12 Mbps

=====

Fenomenologia dell'attacco sul Target:

1. **Analisi di Rete (Wireshark):** La cattura del traffico tramite filtro `udp.port == 5005 or icmp` ha evidenziato un comportamento simmetrico e bidirezionale dello stack di rete. Per ogni pacchetto UDP da 1 KB inviato da Kali, Metasploitable (rilevando che la porta 5005/UDP era chiusa e che nessun servizio era in ascolto) ha risposto inviando un pacchetto di errore **ICMP Type 3, Code 3 (Destination Unreachable - Port Unreachable)** verso l'IP di Kali.
2. **Analisi delle Risorse CPU (Metasploitable):** Durante i 22,89 secondi di attacco, l'utility `top` sul target ha registrato una saturazione quasi totale della CPU. L'analisi dei processi ha confermato che il consumo di risorse non era legato ad applicazioni nello spazio utente (*user space*), bensì al processo del kernel **ksoftirqd**. Questo thread di sistema è stato congestionato dalla gestione ininterrotta degli interrupt software (`softIRQ`) necessari per elaborare i pacchetti UDP in ingresso e generare i relativi pacchetti ICMP di risposta in uscita.

5. Strategie di Rilevamento e Mitigazione (Cyber Defense)

A. Rilevamento tramite IDS (Snort / Suricata)

Per identificare tempestivamente questo scenario di anomalia prima della saturazione dei servizi, si propone l'adozione di una firma personalizzata basata su soglia (*thresholding*) per i sistemi di rilevamento intrusioni:

Plaintext

```
alert udp $EXTERNAL_NET any -> $HOME_NET 5005 (msg:"Rilevato potenziale UDP Flood su porta 5005"; threshold:type threshold, track by_src, count 2000, seconds 2; sid:1000001; rev:1;)
```

Spiegazione: Questa firma genera un alert di sicurezza se un singolo indirizzo IP sorgente esterno invia più di 2.000 pacchetti UDP verso la porta 5005 interna in un intervallo di tempo pari a 2 secondi.

B. Mitigazione tramite Firewall Host-Based (iptables)

Per neutralizzare l'efficacia dell'attacco e preservare le risorse di calcolo del server, è possibile intervenire sulla configurazione delle tabelle dei filtri del kernel Linux.

- **Drop Silenzioso dei pacchetti (Interruzione del ciclo ICMP):** Applicando la seguente regola su Metasploitable, il firewall scarta i pacchetti prima che vengano elaborati dallo stack di rete:

Bash

```
sudo iptables -A INPUT -p udp --dport 5005 -j DROP
```

Impatto: Il sistema smette di inviare risposte ICMP "Port Unreachable". Wireshark mostrerà il traffico UDP in una sola direzione (unidirezionale da Kali a Metasploitable). Il carico di lavoro di ksoftirqd sulla CPU del target si riduce drasticamente di oltre il 50%.

- **Prevenzione tramite Limitazione di Frequenza (Rate Limiting):** Per proteggere l'infrastruttura da scansioni massive o flood su porte UDP generiche senza bloccare interamente il protocollo, si impone una soglia massima di pacchetti accettati:

Bash

```
sudo iptables -A INPUT -p udp -m limit --limit 50/s --limit-burst 100 -j ACCEPT  
sudo iptables -A INPUT -p udp -j DROP
```

Impatto: Vengono accettati al massimo 50 pacchetti UDP al secondo, prevenendo la saturazione della banda e delle risorse di calcolo.

6. Conclusioni

L'esercitazione ha dimostrato con successo la dinamica di un attacco DoS a livello di trasporto. Nonostante lo script abbia operato a una banda moderata di **34,12 Mbps**, l'assenza di difese attive sul target ha evidenziato come l'overhead di gestione del protocollo (generazione dei messaggi ICMP d'errore) possa rappresentare un vettore di criticità persino superiore al volume di traffico stesso.

L'implementazione delle regole di filtraggio iptables testate nel corso dell'attività rappresenta la soluzione ottimale ed economica per mitigare questa specifica classe di minacce a livello host.

Elenco degli Allegati Richiesti per la Consegna

- **Allegato 1:** Screenshot del terminale di Kali Linux al termine dell'esecuzione del file `udp_stress_test.py` contenente lo specchio riassuntivo delle metriche.

```

SIMULATORE UDP FLOOD - INTERFACCIA ESERCITAZIONE

[?] Inserisci l'IP della macchina target: 192.168.50.101
[?] Inserisci la porta UDP target: 5005
[?] Inserisci il numero totale di pacchetti da inviare (da 1 KB cad.): 100000

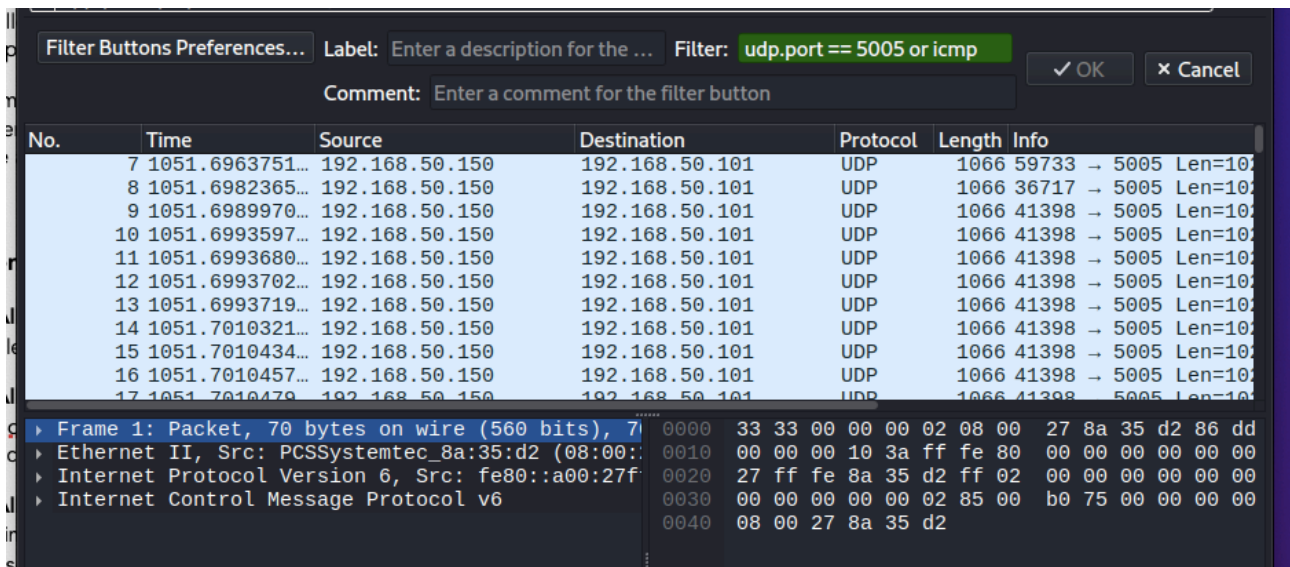
[*] Bersaglio impostato: 192.168.50.101:5005
[*] Dimensione pacchetto: 1 KB (1024 byte)
[*] Totale pacchetti pianificati: 100,000
[*] Avvio della simulazione in corso...

RISULTATI E METRICHE DI CARICO

[*] Pacchetti inviati con successo: 100,000 su 100,000
[*] Tempo impiegato: 22.8975 secondi
[*] Volume totale dei dati inviati: 97.66 MB
[*] Velocità media di invio: 4367.29 pacchetti/secondo
[*] Larghezza di banda teorica: 34.12 Mbps

```

- **Allegato 2:** Screenshot della sessione di cattura attiva su Wireshark, filtrata per `udp.port == 5005 or icmp`, in cui sia visibile la corrispondenza temporale tra le richieste UDP e le risposte ICMP di errore.



- **Allegato 3:** Screenshot del comando `top` su Metasploitable eseguito durante la simulazione, con evidenziato il consumo percentuale di CPU del processo `ksoftirqd`.

